BIGTREETECH

Pad5 V2.0

User Manual

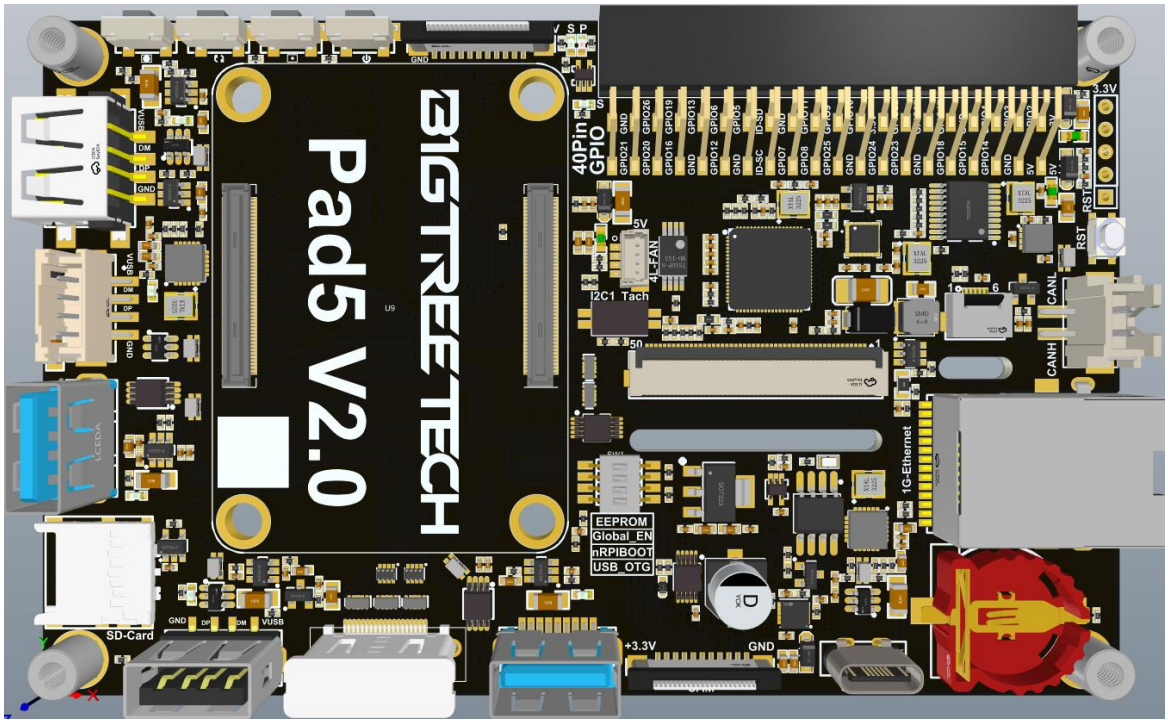


Table of Contents

Revision Log 4
1. Product Overview 5
1.1. Product Feature 5
1.2. Specifications 5
2. Peripheral Interfaces 6
2.1 Interface Diagram 6
2.2 Feature Overview 7
3. Feature Details 7
3.1. 40 Pin GPIO 7
3.2. CAN Interface 8
3.3. Fan Interface 8
3.4. RTC Circuit 8
3.5. MIPI Interfaces 8
3.6. Buttons 9
4. Use with the Raspberry Pi CM4/CM5 11
4.1. Download the System Image 11
4.2. Download and Install Raspberry Pi Imager 11
4.3. Flashing the System 11
4.3.1 Flashing to a Micro SD Card 11
4.3.2 Flashing to eMMC（On eMMC versions, the system cannot boot from the Micro SD card.） 16
4.4. System Configuration 17
4.4.1 DSI Display 17
4.4.2 CAM Camera 18
4.4.3 RTC 19
4.4.4 CANBus 20
5. Used with the BIGTREETECH CB2 21

5.1. Download the System Image 21
5.2. Download and install the balenaEtcher 21
5.3. Flashing the System 21
5.3.1 Write System to MicroSD Card 21
5.3.2 Writing System onto eMMC 22
5.3.2.1 Using RKDevTool to Write the eMMC (Windows) 22
5.3.2.2 Writing System onto eMMC Using a MicroSD Card 25
5.4. Erasing eMMC 26
5.4.1 Using UMS to Erase eMMC(Windows) 26
5.4.2 Using RKDevTool to Erase eMMC (Windows) 27
5.4.3 Erasing eMMC After Booting from MicroSD Card 27
5.5. System Configuration 28
5.5.1 DSI Display 28
5.5.2 CAM Camera 30
5.5.3 RTC 30
6. Used with the BIGTREETECH CB1 30
6.1. Used with the BIGTREETECH CB1 30
6.2. Download and Install balenaEtcher 31
6.3. Flashing the System 31
6.3.1 Flashing the System 31
6.3.2 Flashing to eMMC 32
6.3.2.1 Use balenaEtcher to flash the system image to eMMC 32
6.3.2.2 Writing the System to eMMC from a Micro SD Card 36
6.4. Erasing eMMC 38
6.4.1 Erase eMMC Using UMS (Windows) 38
6.4.2 Erase eMMC After Booting from a Micro SD Card 38
7. Notes 38

Revision Log

Version	Revision	Date
01.00	Initial Version	2025/12/29

1. Product Overview

The BIGTREETECH Pad5 V2.0 is an expansion display board compatible with CM-series modules, featuring special enhancements for the Raspberry Pi CM5 for easier operation.

1.1. Product Feature

1. Compatible with multiple SoC compute modules, including CM5, CM4, CB2, and CB1.
2. The onboard USB OTG port allows convenient image flashing for compute modules with eMMC storage.
3. Supports USB 3.0 communication when used with the CM5.
4. Includes a reserved XH2.54 2-pin CAN interface for DIY applications.
5. Equipped with a 5-inch IPS capacitive touchscreen for improved display quality and a better user experience.
6. All interfaces feature ESD protection for enhanced product stability.
7. Supports one-button power on/off when used with the CM5, making operation more convenient.
8. Features an onboard host-side USB hub: one channel connects to the UART port via CH340E, while another connects directly to USB OTG, making image flashing and SoC debugging more convenient.
9. Offers multiple USB interface options for greater flexibility in DIY applications.

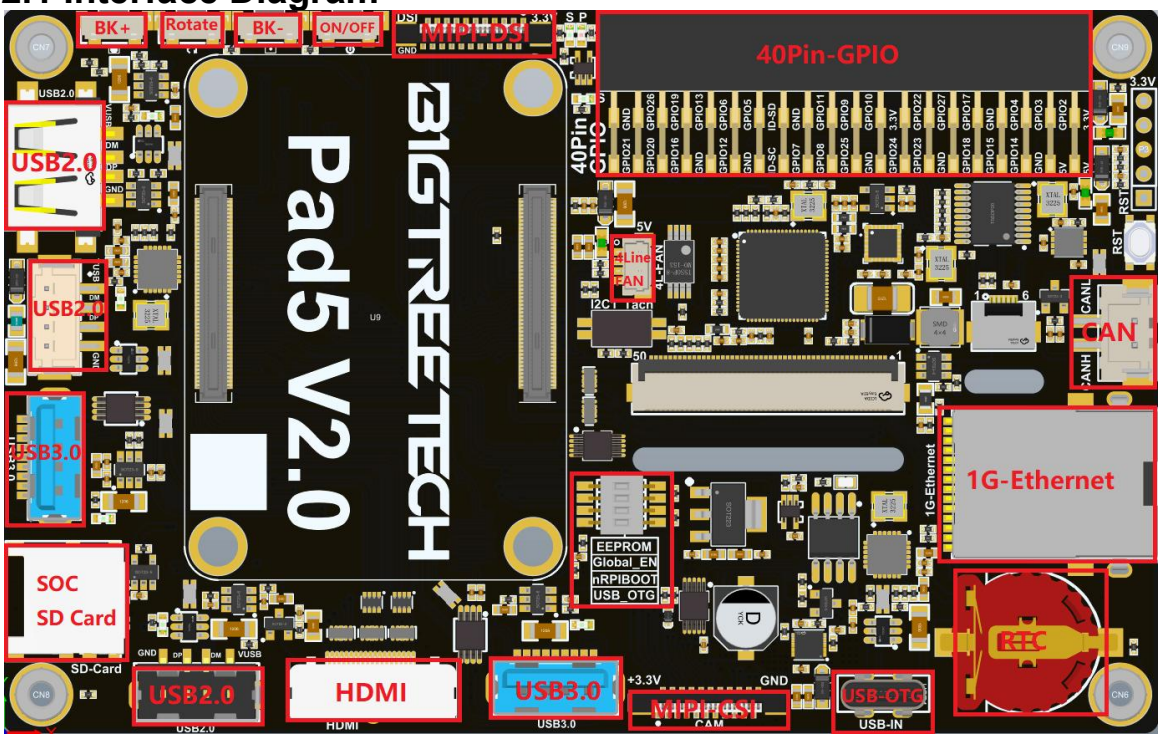
1.2. Specifications

1. Display Size：21.1mm × 75.9mm
2. Screen：5-inch 800 × 480 capacitive touchscreen
3. USB Host Ports：3 × USB 2.0 (5V/1.5A output), 2 × USB 3.0 (5V/2.7A output)
4. USB OTG Port：USB Type-C
5. Ethernet Port：Gigabit Ethernet
6. Power Input：USB Type-C, 5V/5A (PD supported)

7. Logic Voltage：3.3V
8. Expansion Functions and Interfaces：MIPI-DSI, MIPI-CSI, 40-pin GPIO, CAN, SD card (available only when eMMC is not in use), 4-pin FAN, Micro HDMI, RTC, etc.
9. Buttons：BK+ (Backlight +), Rotate, BK− (Backlight −), ON/OFF (hold BK+ or BK− to adjust faster)

2. Peripheral Interfaces

2.1 Interface Diagram



2.2 Feature Overview

The table below summarizes the features supported by each compatible core board: √= Supported, ×= Not supported.

	CM5	CM4	CB2	CB1
Ethernet	√Gigabit	√Gigabit	√Gigabit	√100Mbps
CAN	√	√	×	×
40Pin-GPIO	√	√	√	√
MIPI-DSI	√ (DPHY1)	√ (DSI1)	√ (DSI)	×
5' TFT Display&Touch	√	√	√	√
Power switch button	√	×	×	×
BK+、Rotate、BK-	√	√	√	√
USB2.0x3	√	√	√	√
USB3.0x2	√	×	×	×
SOC-SD Card	√	√	√	√
HDMI	√	√	×	×
5V-4Line FAN	√	×	×	×
MIPI-CSI	√ (DPHY0)	√ (CAM1)	√ (CSI)	×
USB-OTG	√	√	√	√
RTC	√	√	√	×

3. Feature Details

3.1. 40 Pin GPIO

The diagram shows the GPIO pinout mapping for different core boards on the 40-pin header.

3.2. CAN Interface

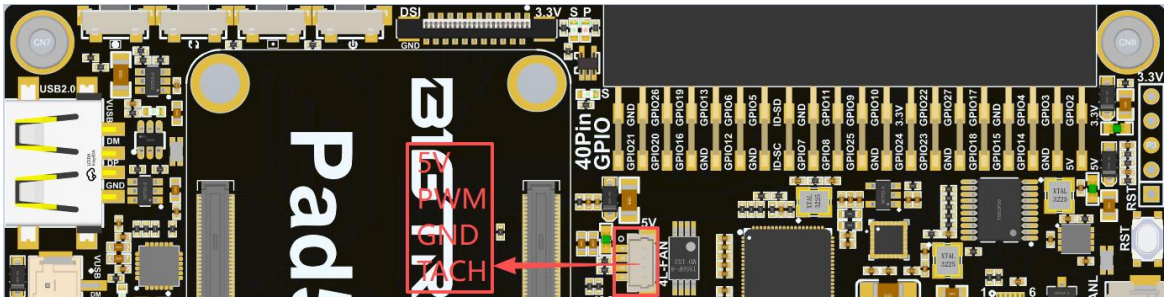
The Pad5 supports CAN communication when used with CM5 or CM4.

This is achieved by converting the SoC's SPI signals to CAN signals.

SPI to CAN Mapping: CS-GPIO8，MISO-GPIO9，MOSI-GPIO10，SCK-GPIO11，INT-GPIO24

3.3. Fan Interface

The Pad5 supports a 4-wire fan for active cooling when used with CM5.
Connector type: ZX-SH1.0 4P.



3.4. RTC Circuit

The Pad5 provides an RTC circuit when used with CM5, CM4, or CB2, using the SoC's I2C interface to control a PCF8563 clock chip.

Note: Before using the RTC, install a CR1220 3V coin cell battery with the logo side facing up; recommended brands include Panasonic, Lithium Cell, and similar.

3.5. MIPI Interfaces

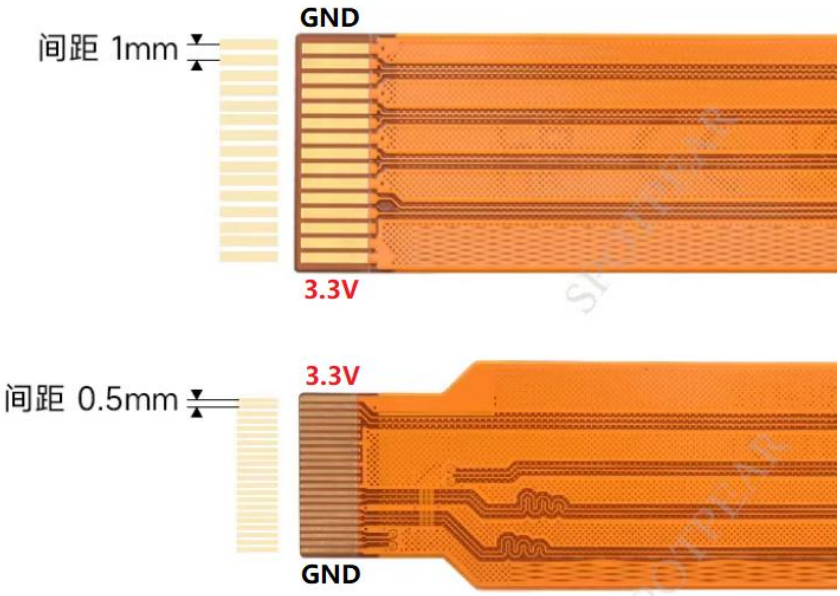
The Pad5 supports MIPI-DSI (display) and MIPI-CSI (camera) when used with CM5, CM4, or CB2. On the CM5, both MIPI interfaces can be used interchangeably, with DPHY0 and DPHY1 functioning as either DSI or CSI. On the CM4 and CB2, the CSI interface connects only to a camera, while the DSI interface connects only to a DSI display.

Important: Use the correct cable for each device—do not use a camera cable for the display or vice versa—and ensure the gold fingers are properly aligned with the connector and the latch is secured.

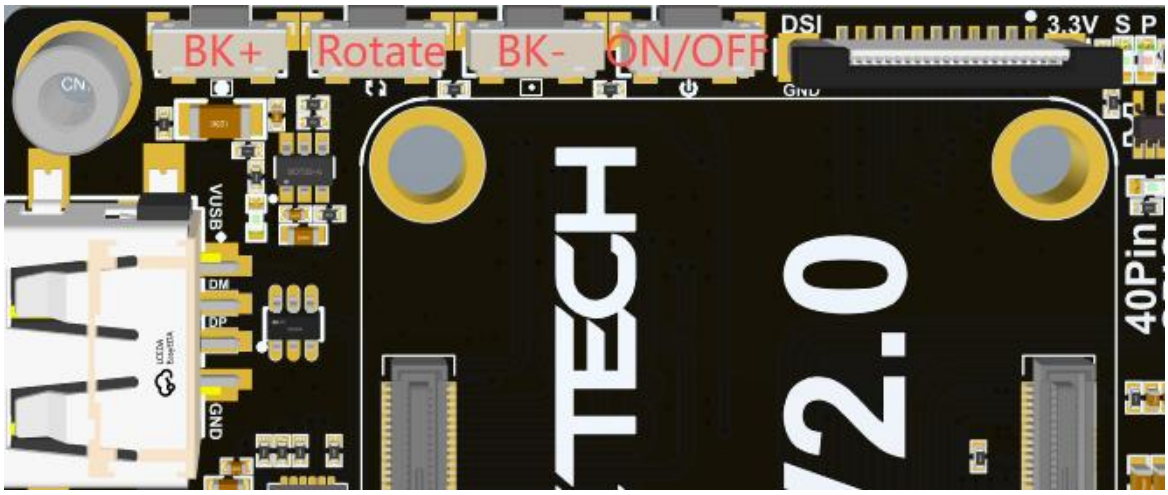
DSI Cable：

CSI ：

Pin Sequence：



3.6. Buttons



BK+: Increase backlight brightness (0–100%). Each click increases by 1%; long press adjusts faster.

Rotate: Rotate display 180°.

BK-: Decrease backlight brightness (0–100%). Each click decreases by 1%; long press adjusts faster.

ON/OFF: Power switch (supported only with Raspberry Pi CM5).

4. Use with the Raspberry Pi CM4/CM5

4.1. Download the System Image

Download the appropriate system image based on your requirements, such as the official Raspberry Pi image or the Mainsail system image.
Official Raspberry Pi OS images:
https://www.raspberrypi.com/software/operating-systems
Mainsail: https://github.com/mainsail-crew/MainsailOS/releases

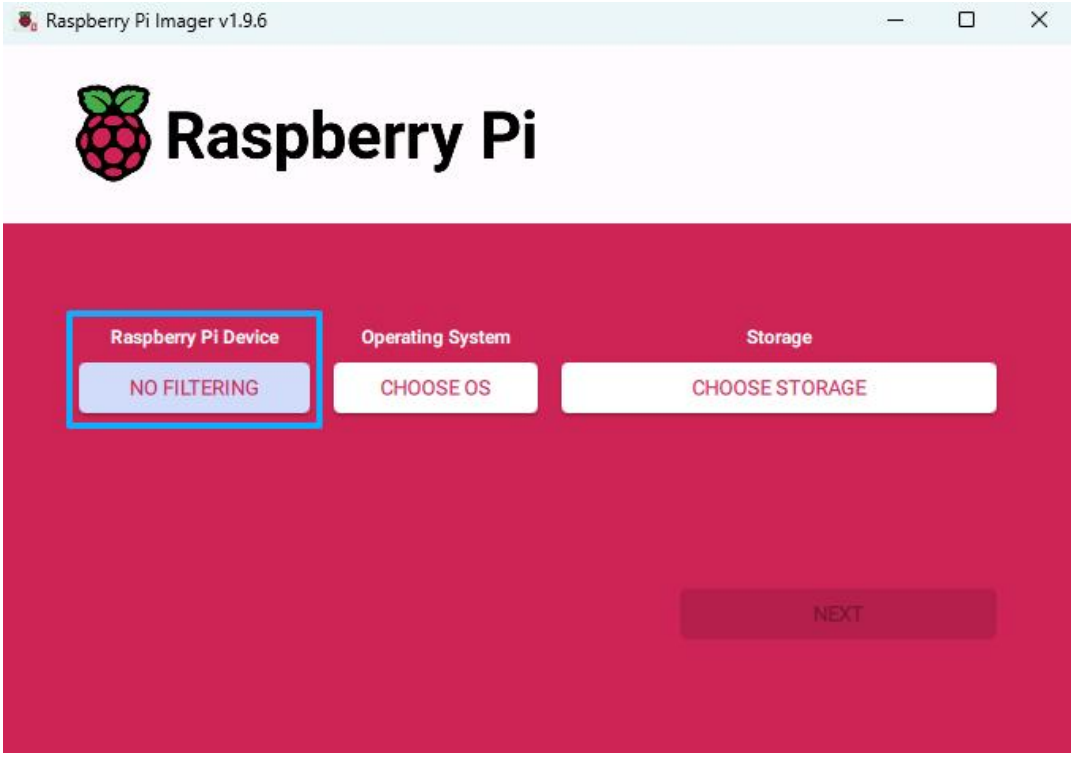
4.2. Download and Install Raspberry Pi Imager

Download and install the official Raspberry Pi flashing tool from the Raspberry Pi website:https://www.raspberrypi.com/software/

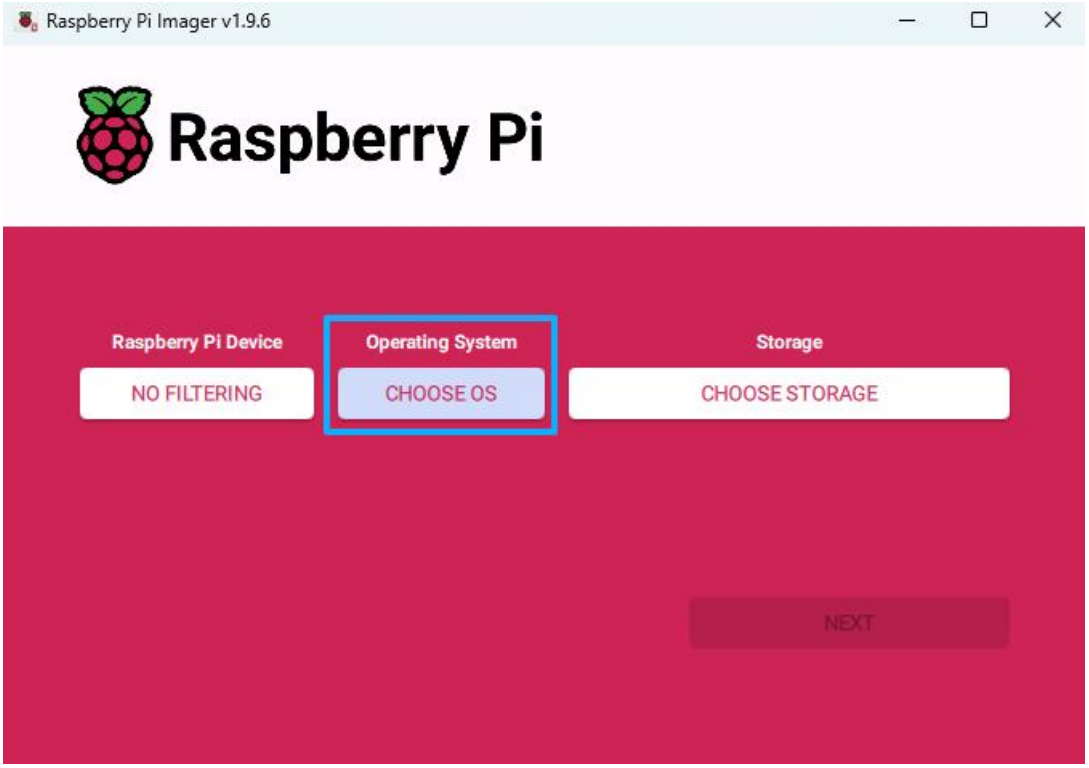
4.3. Flashing the System

4.3.1 Flashing to a Micro SD Card

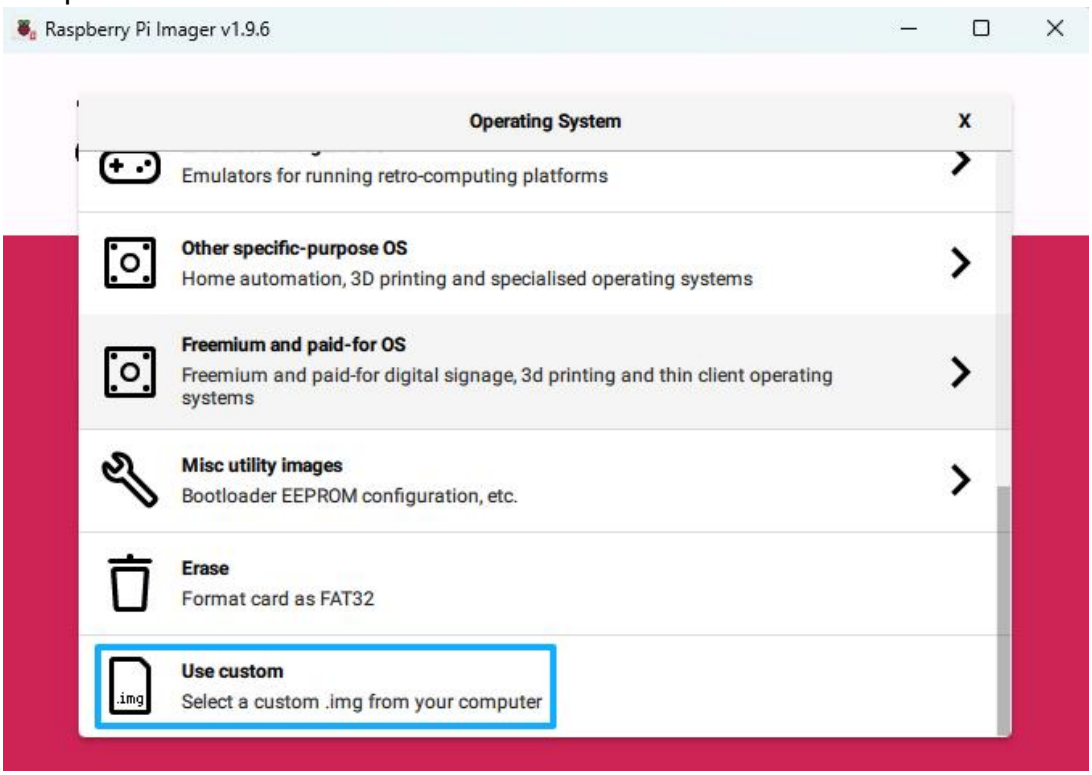
1. Insert the Micro SD card into your computer using a card reader.
2. Set the device to“NO FILTERING”.



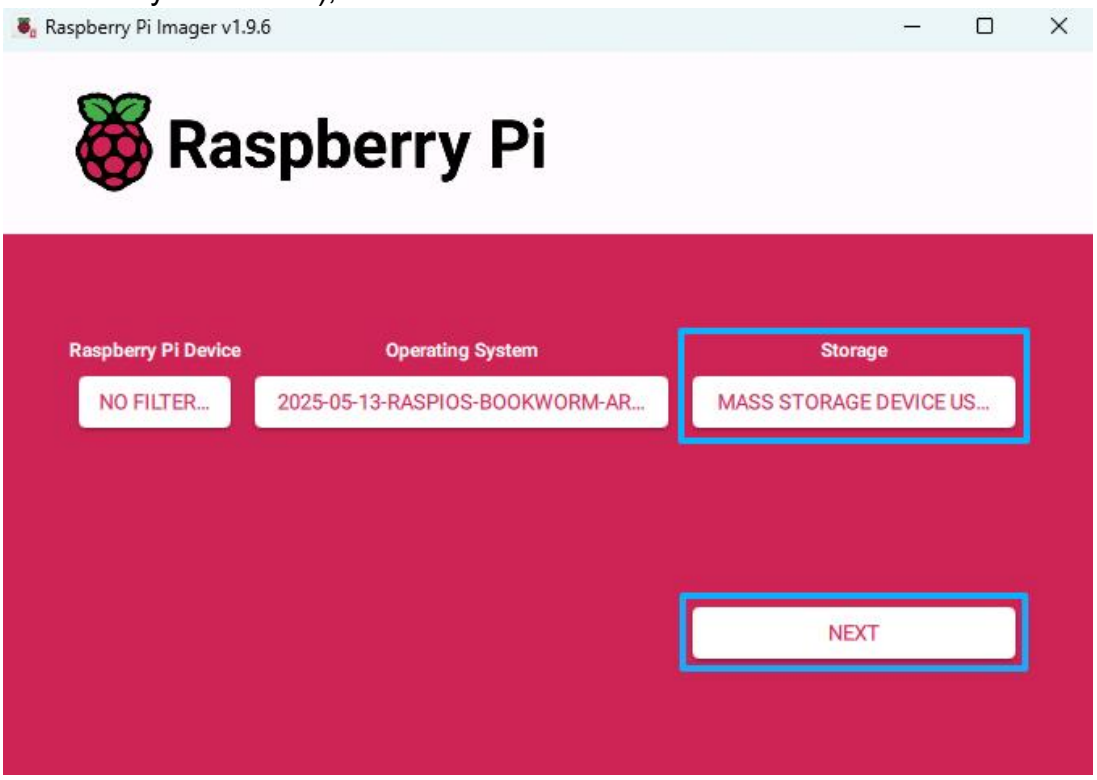
3. Select the operating system.



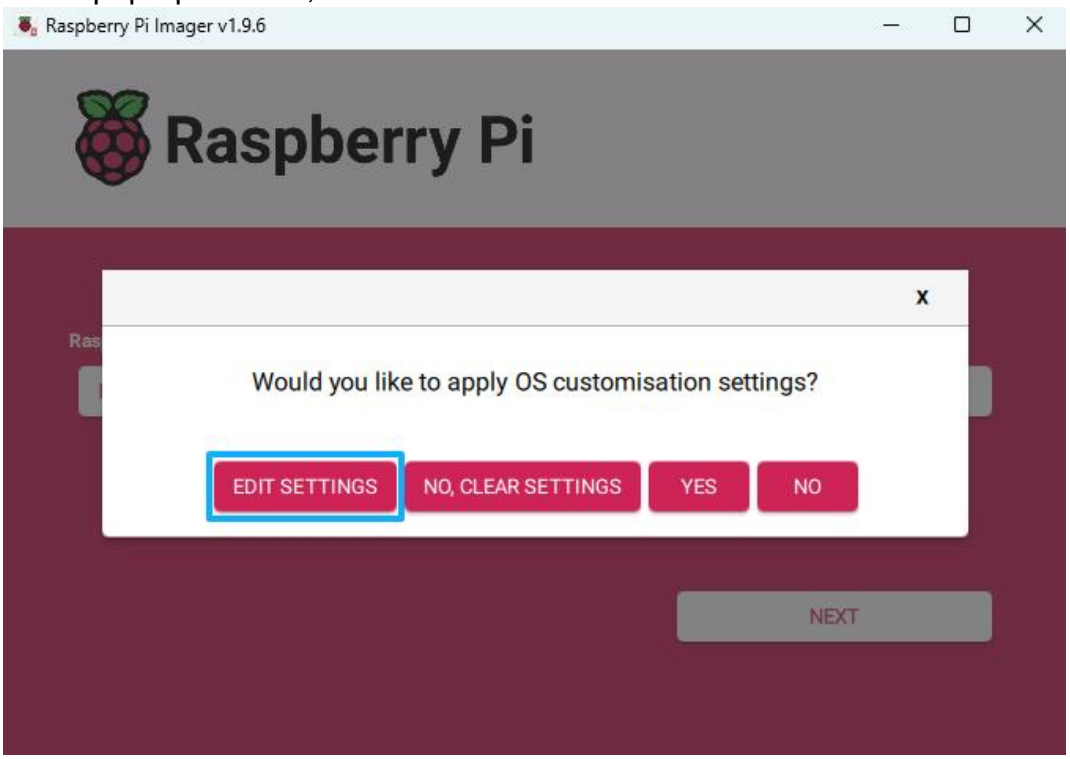
4. Choose “Use custom”, then select the system image file downloaded to your computer.



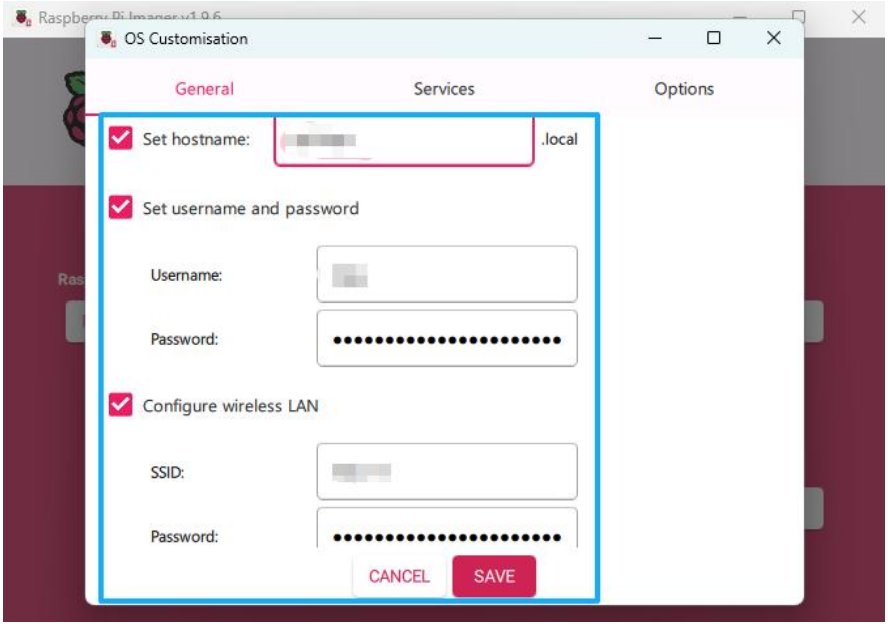
5. Select the Micro SD card to be flashed (flashing the image will format the card, so ensure the correct drive is selected; otherwise, data on another storage device may be erased), then click Next.



6. In the pop-up window, select “EDIT SETTINGS”.

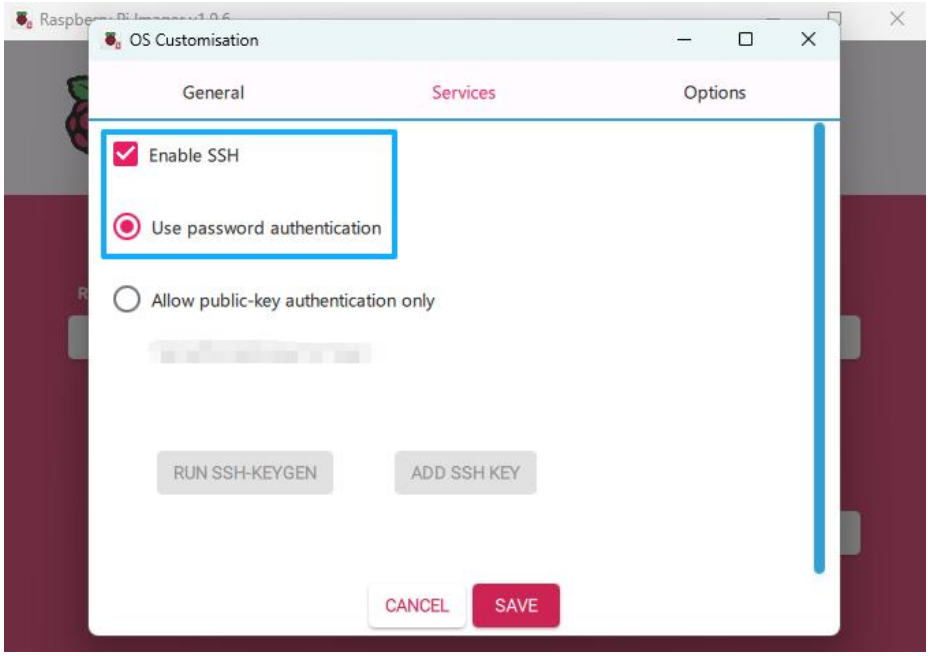


7. Under `General`, configure the following:
   Set hostname: raspberrypi.local // Set hostname. Default hostname:raspberrypi.local
   Set username and password // Set username and password. Default username: pi Default password: raspberry
   Configure wireless LAN // Set the Wi-Fi SSID and password.

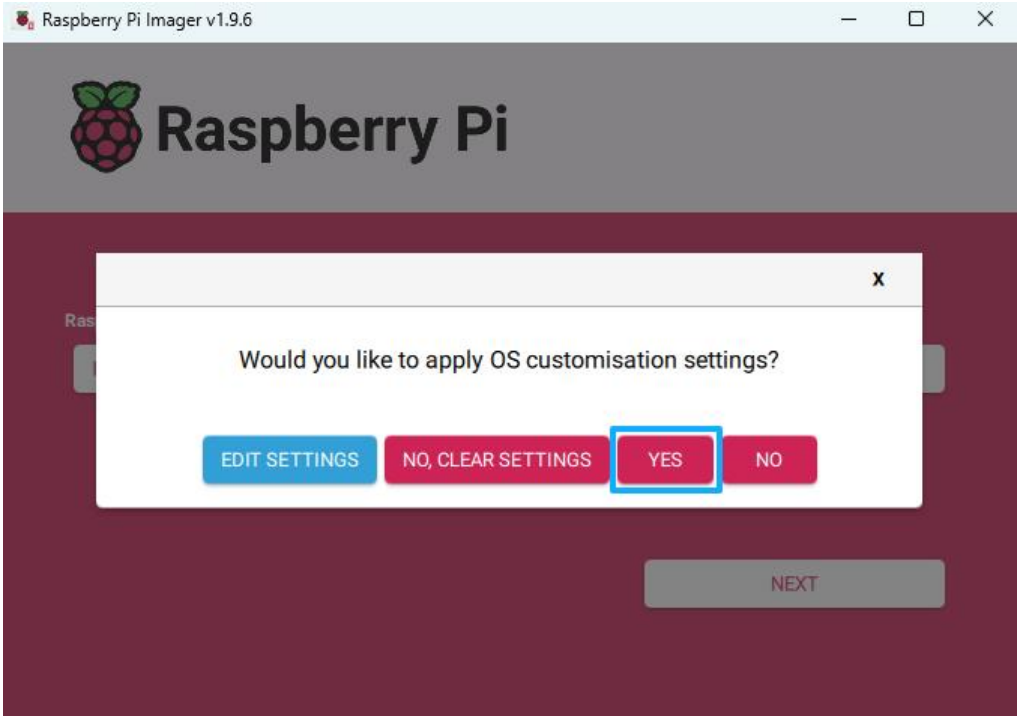


8. Under `Services`, configure the following:
   Enable SSH
   Use password authentication

Then click `SAVE` to store the settings.



9. Click `YES` to start flashing the system image.

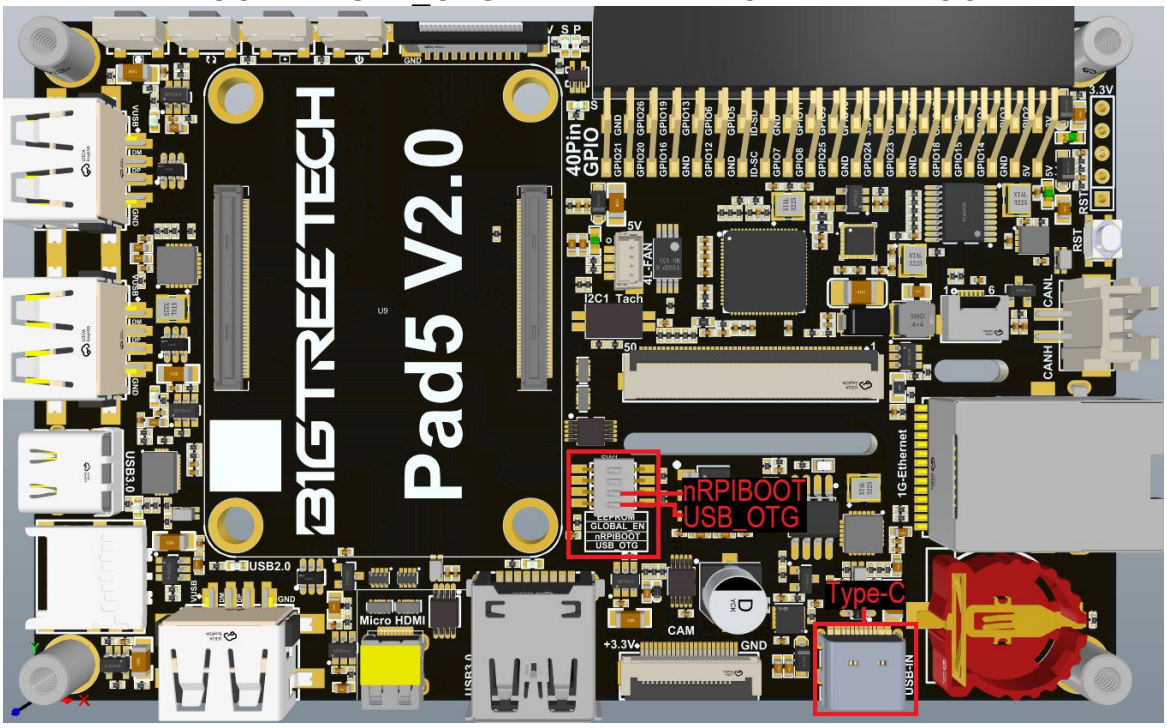


10. Wait for the flashing process to complete.

4.3.2 Flashing to eMMC（On eMMC versions, the system cannot boot from the Micro SD card.）

1. Install the rpiboot Utility
   Windows: Download and install rpiboot:
   http://github.com/raspberrypi/usbboot/raw/master/win32/rpiboot_setup.exe
   Mac/Linux: Refer to the official usbboot build instructions:
   https://github.com/raspberrypi/usbboot#building

2. Set the nRPIBOOT and USB_OTG DIP switches to ON to enter BOOT mode.



3. Connect the Type-C port to your computer's USB port (To avoid insufficient USB power from the computer, it is recommended to use a powered USB hub or provide external power through the 5V pins on the 40-pin GPIO header), run: `sudo ./rpiboot（Mac/Linux）` or `rpiboot.exe（Windows）`, then the eMMC will then be recognized by the computer as a USB mass storage device (If rpiboot reports an error at this stage, try reconnecting the USB cable).

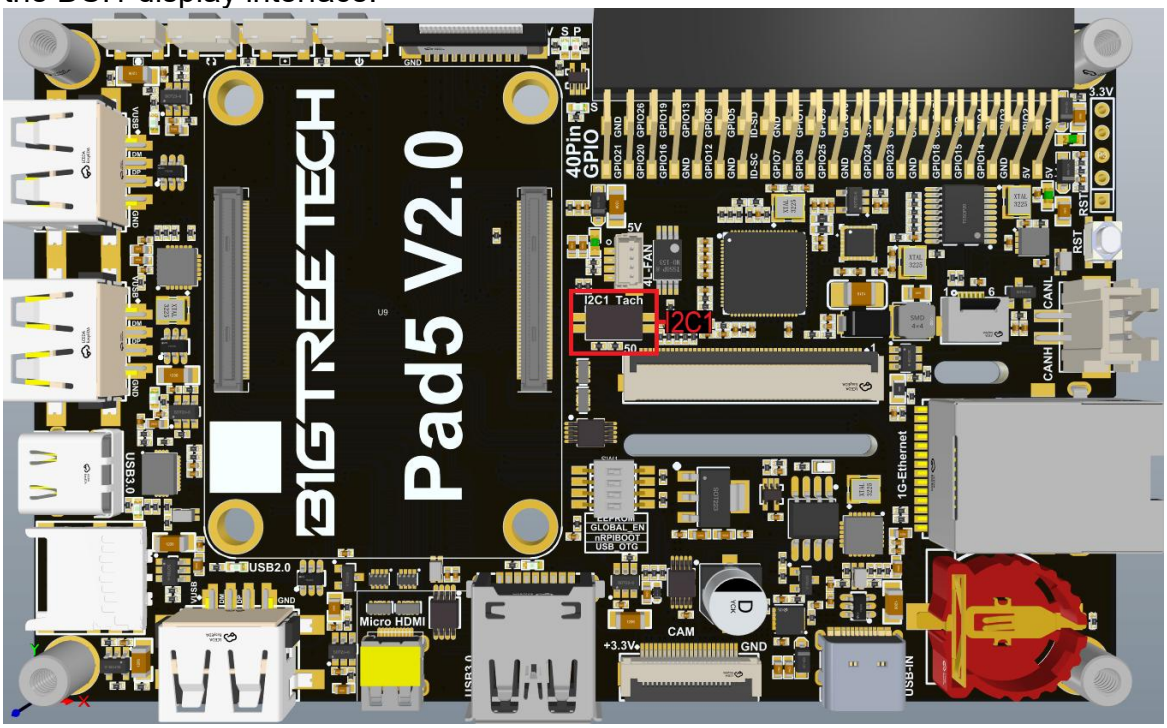
4. Use Raspberry Pi Imager to flash the system image; the procedure is the same as flashing to a Micro SD card.

5. After flashing is complete, power off the device, switch nRPIBOOT and USB_OTG back to OFF, then power it on again to enter normal operating mode.

4.4. System Configuration

4.4.1 DSI Display

1. Set both 2-pin I2C DIP switches to ON; this enables the touch signal line for the DSI1 display interface.



2. Edit the `/boot/firmware/config.txt` file as follows.
   For CM5

   ```
   [cm5]
   dtoverlay=vc4-kms-v3d
   dtoverlay=vc4-kms-dsi-generic
   dtoverlay=edt-ft5406,i2c_csi_dsi
   ```

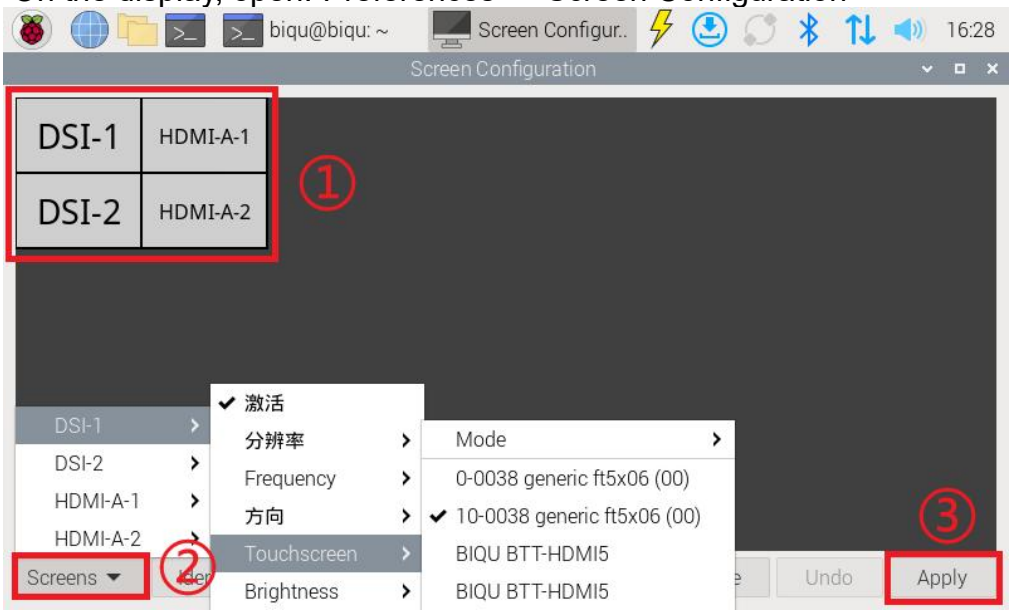
   For CM4

   ```
   [cm4]
   dtoverlay=vc4-kms-v3d
   dtoverlay=vc4-kms-dsi-generic
   dtoverlay=edt-ft5406,i2c_csi_dsi0
   ```

   Restart the device after editing the configuration.

3. At this point, both the built-in display and the external DSI display should function normally. However, touch input must be remapped when using multiple displays.

   3.1. On the display, open:`Preferences`->`Screen Configuration`



   3.2. ① Drag and arrange “DSI-1”, “DSI-2”, “HDMI-A-1” and “HDMI-A-2” as needed; the physical display layout will follow this arrangement, and the displays can also be fully overlapped to show identical content.

   3.3. ② Assign each touch device to its corresponding display so that, after configuration, each touch device controls only its assigned display.

   3.4. ③ Apply the settings and click Apply for the configuration to take effect.

4.4.2 CAM Camera

1. Edit ` /boot/firmware/config.txt` as follows.
   For CM5

   ```
   [cm5]
   dtoverlay=ov5647,cam0
   ```

   For CM4

   ```
   [cm4]
   dtoverlay=ov5647
   ```

   Restart the device. The OV5647 camera is used here as an example. If you are using a different camera model, replace the overlay with the appropriate one.

2. Testing the Camera
   List available cameras:

```
rpicam-hello --list-camera
```

Set the display output:

```
export DISPLAY=:0.0
```

Start the camera preview(do not run as root):

```
rpicam-hello --timeout 0 --camera 0
```

For more detailed usage, refer to the Raspberry Pi Camera Software Documentation:

https://www.raspberrypi.com/documentation/computers/camera_software.html

4.4.3 RTC

1. The onboard RTC chip is `PCF8563`; a CR1220 coin cell battery must be installed for the RTC to continue operating when power is disconnected.
2. Edit `/boot/firmware/config.txt` as follows.
   For CM5

   ```
   [cm5]
   dtparam=i2c_vc=on
   dtoverlay=i2c-rtc,pcf8563,i2c_csi_dsi0
   ```

   For CM4

   ```
   [cm4]
   dtparam=i2c_vc=on
   dtoverlay=i2c-rtc,pcf8563,i2c_csi_dsi
   ```

   Restart the device after editing the configuration.
3. After booting, run:

   ```
   dmesg | grep rtc
   ```

   Check which RTC device number has been assigned; if pcf8563 is mounted as rtc0, use rtc0 in the following commands, and if it is mounted as rtc1, use rtc1 instead.
4. RTC Test Example: Using rtc1 as an example.

   Write the system time to RTC:

   ```
   sudo hwclock -w -f /dev/rtc1
   ```

   Set the system time from RTC:

   ```
   sudo hwclock -s -f /dev/rtc1
   ```

   Read and display the RTC time:

```
sudo hwclock -r -f /dev/rtc1
```

4.4.4 CANBus

1. Edit `/boot/firmware/config.txt` as follows; the same configuration applies to both CM5 and CM4.

   ```
   [all]
   dtparam=spi=on
   dtoverlay=mcp2515-
   can0,oscillator=12000000,interrupt=24,spimaxfrequency=10000000
   ```

   Restart the device.

2. After booting, run:

   ```
   dmesg | grep -i '\(can\|spi\)'
   ```

   Check whether the MCP2515 has been initialized successfully; a normal response will look similar to:

   ```
   [ 8.680446] CAN device driver interface
   [ 8.697558] mcp251x spi0.0 can0: MCP2515 successfully initialized.
   [ 9.482332] IPv6: ADDRCONF(NETDEV_CHANGE): can0: link becomes
   ready
   ```

5. Used with the BIGTREETECH CB2

5.1. Download the System Image

Only use the image provided in the link:
https://github.com/bigtreetech/CB2/releases

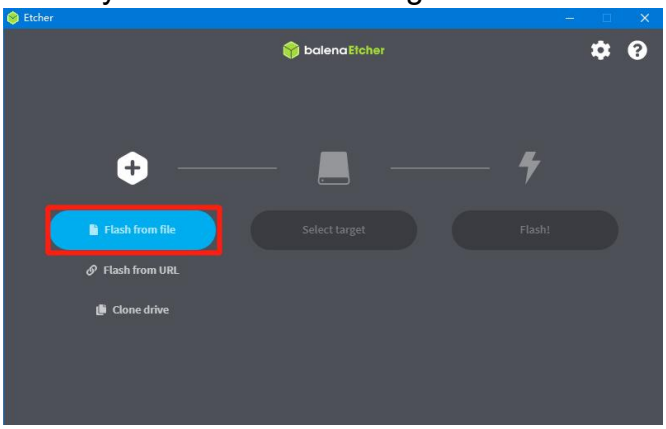
5.2. Download and install the balenaEtcher

https://www.balena.io/etcher/

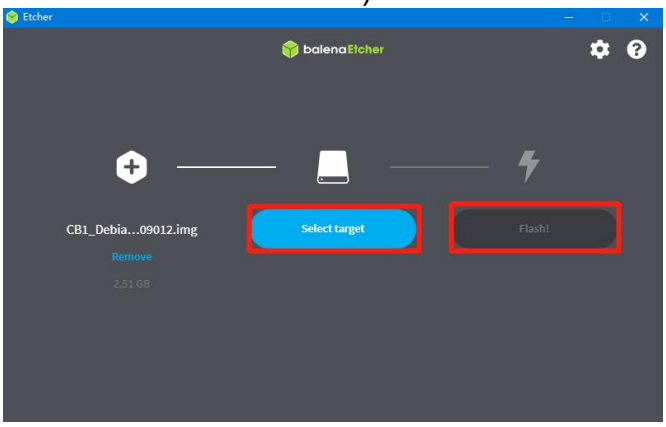
5.3. Flashing the System

5.3.1 Write System to MicroSD Card

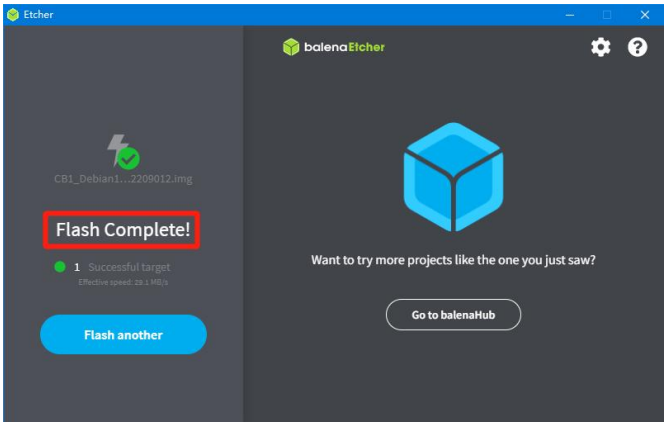
1. Insert the MicroSD card via a card reader.
2. Select your downloaded image.



3. Select the MicroSD card and click "Flash" (WRITE the image will format the MicroSD card. Be careful not to select the wrong storage device, otherwise the data will be formatted).



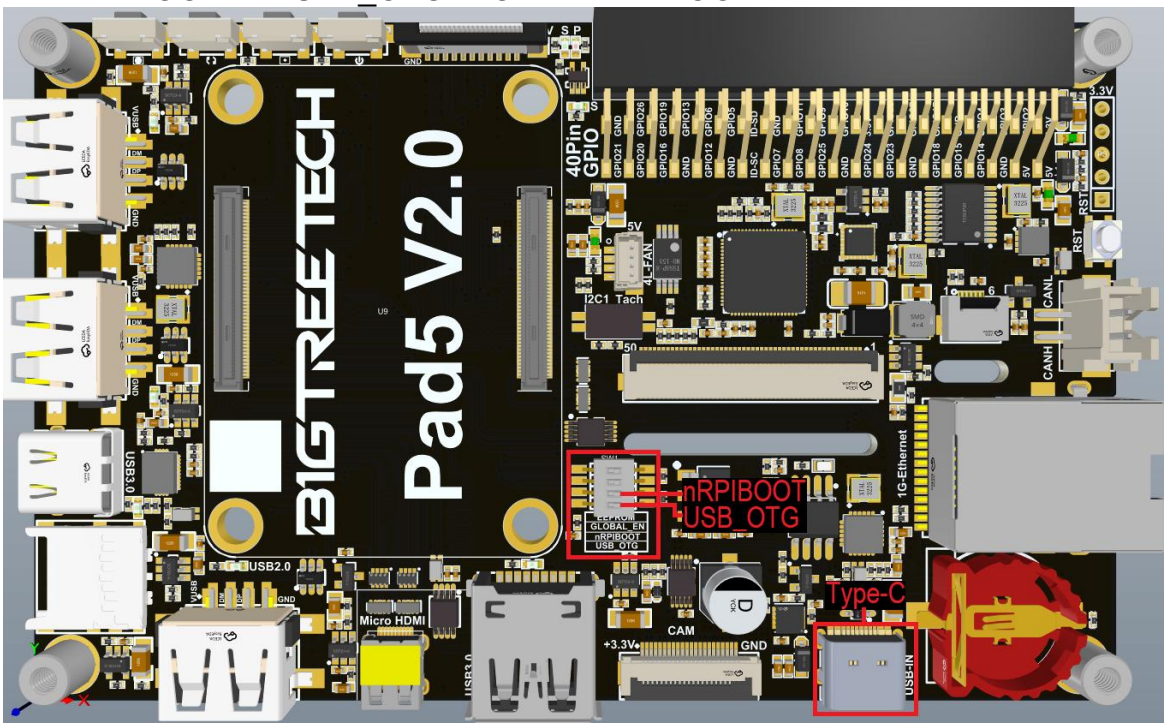
4. Wait for the process to comlpete



5.3.2 Writing System onto eMMC

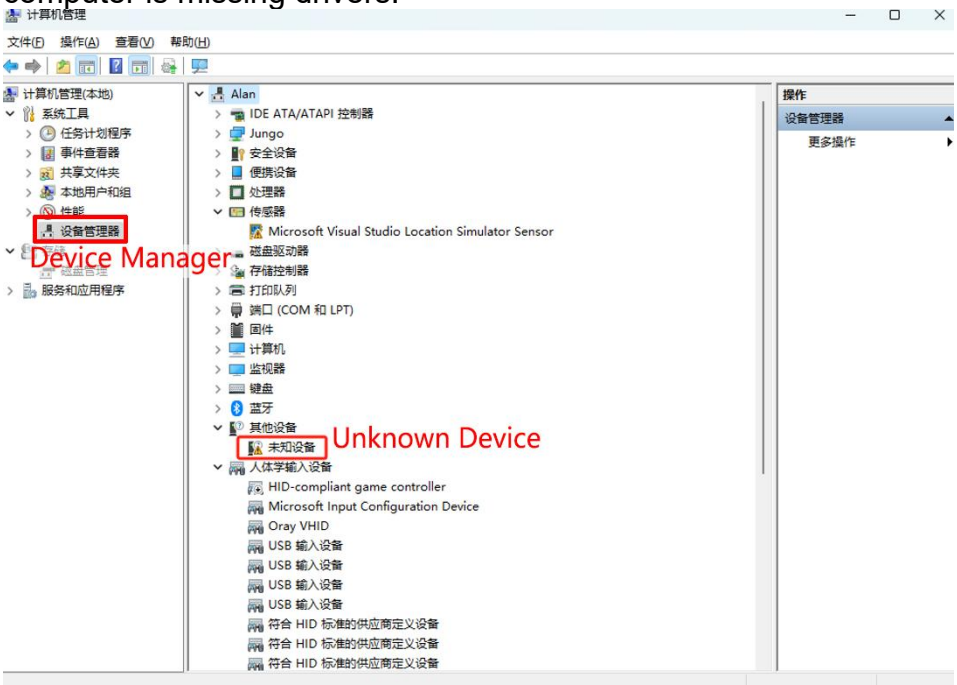
5.3.2.1 Using RKDevTool to Write the eMMC (Windows)

1. Download and unzip RKDevTool(https://github.com/bigtreetech/CB2)from your computer.DO NOT insert a MicroSD card.
2. Set nRPIBOOT and USB_OTG to ON to enter BOOT mode.



3. Connect the Type-C port to your computer's USB port (To avoid insufficient USB power from the computer, it is recommended to use a powered USB hub or provide external power through the 5V pins on the 40-pin GPIO header)
4. Install the driver:

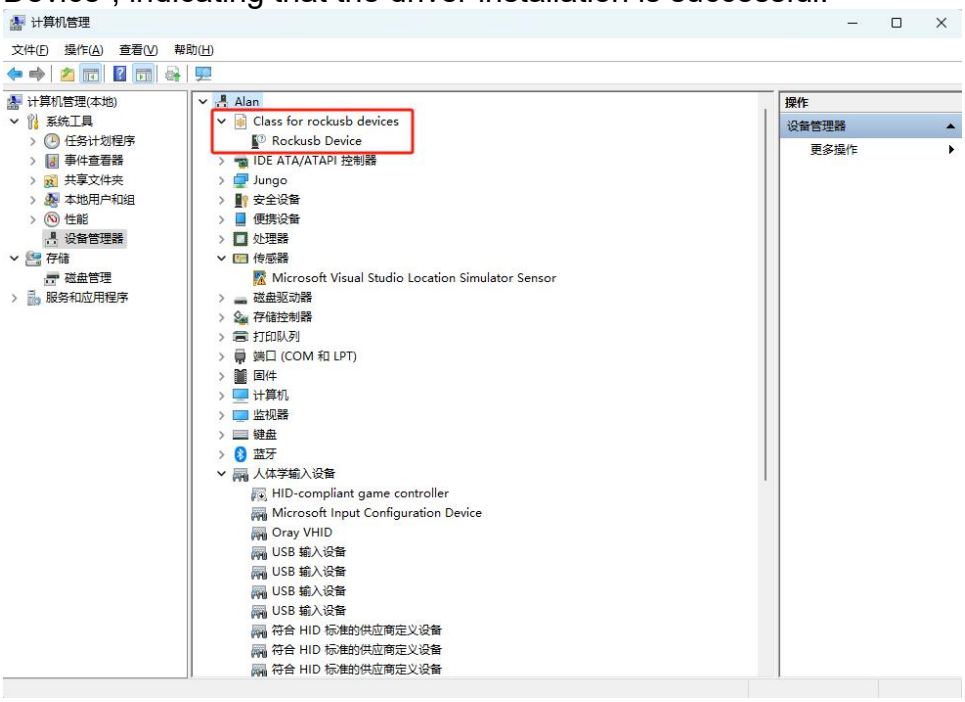
(1) In “Device Manager”, if you see “Unknown Device”, it indicates that the computer is missing drivers.



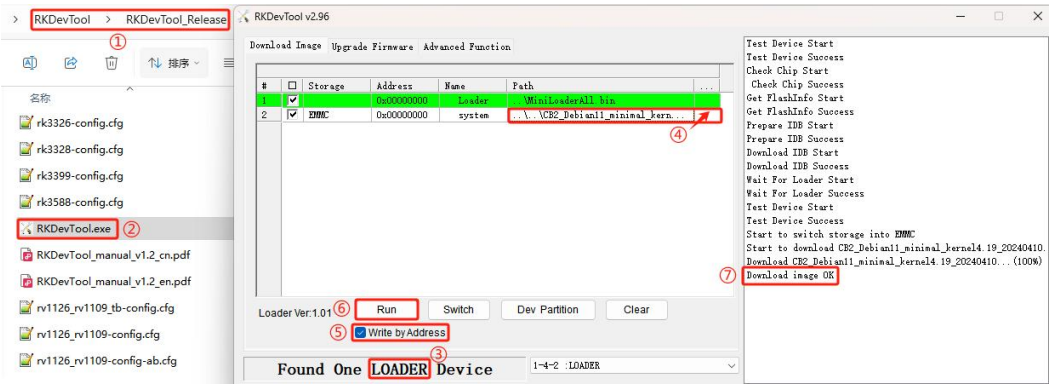
(2) Open the DriverAssistant tool in the downloaded RKDevTool folder, click “①Uninstall Driver", then click “② Install Driver” to ensure that the latest version of the driver is installed.

(3) After the installation is complete, hold down the “Recovery” button, replug the Type-C cable. "Device Manager" should now recognize a “Rockusb

Device", indicating that the driver installation is successful.



5. Open "RKDevTool" software:



Note: The parameters in the software are set by default as shown in the image. Normally,you only need to set the "④actual path of the .img system". If the parameters in your software do not match those in the image, manually adjust them to match.

#	□	Storage	Address	Name	Path
1	☑		0x00000000	Loader	..\MiniLoaderAll.bin
2	☑	EMMC	0x00000000	System	actual path of the .img system

①Find the path where the downloaded RKDevTool is located.

②Open the RKDevTool tool.

③The software will recognize a "LOADER" or "MASKROOM" device.

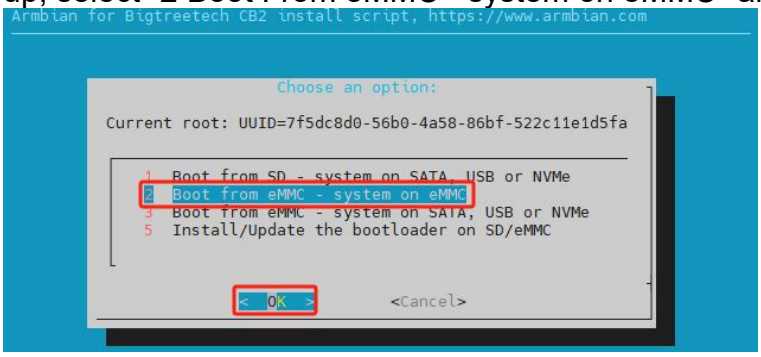
④Select the system to be written (the OS image must be unzipped as a.img file beforehand; RKDevTool does not support directly writing compressed.xz files).
⑤Check "Write by Address"
⑥Click “Run” to start writting the system.
⑦“Download image OK” indicates that the system has been successfully burned.

6. After flashing is complete, power off the board and set both nRPIBOOT and USB_OTG back to OFF. Then power the board on again to enter normal operating mode.

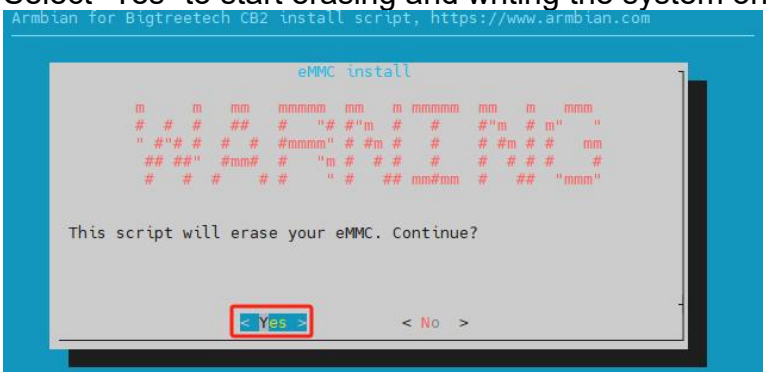
5.3.2.2 Writing System onto eMMC Using a MicroSD Card

1. First, write the system onto the MicroSD card. Then, insert the MicroSD card into the motherboard's card slot and wait for the system to boot.
2. Connect to the system terminal via Ethernet cable, WiFi, or USB to UART, and log in to the system.
   login: biqu
   password: biqu

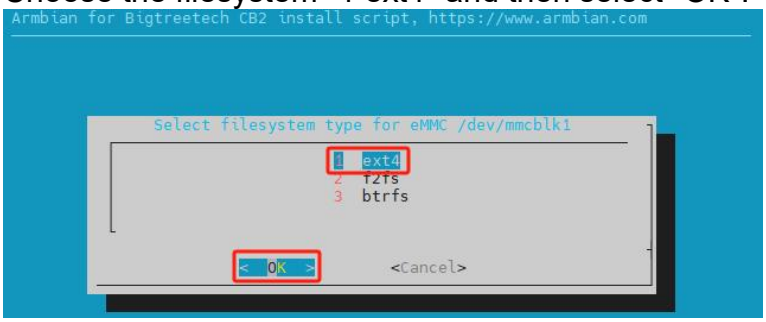
   (1) Execute the command sudo nand-sata-install. In the interface that pops up, select "2 Boot From eMMC - system on eMMC" and then select "OK".



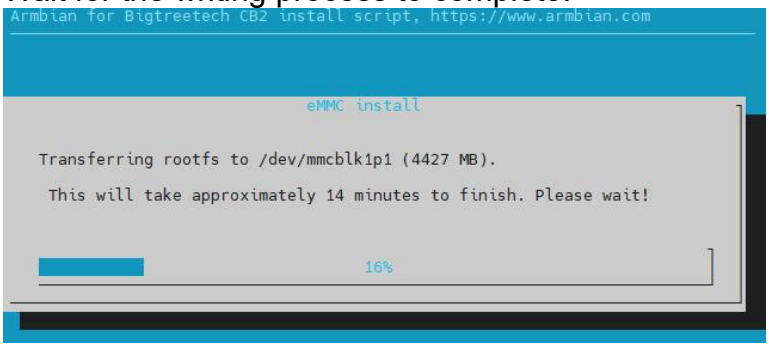
   (2) Select "Yes" to start erasing and writing the system onto the eMMC.



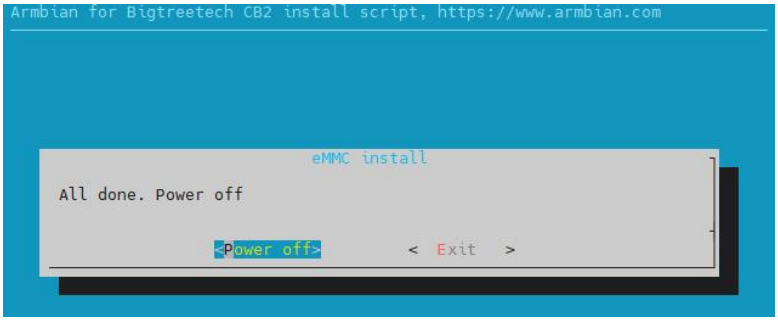
(3) Choose the filesystem "1 ext4" and then select "OK".



(4) Wait for the writing process to complete.



(5) Upon completion, you will be prompted whether to power off. Select “Power off” to shut down the system.



(6) After the system has powered down, disconnect the power supply, remove the MicroSD card, and then reconnect power. The system should now boot from the eMMC.

5.4. Erasing eMMC

When using a MicroSD card as the system card instead, it's best to erase the data on the eMMC to prevent the motherboard from booting from it by mistake.

5.4.1 Using UMS to Erase eMMC(Windows)

If the eMMC has already been flashed with V3.0.1 or later, the computer will recognize it as a UMS device (similar to a USB drive). Compared with Loader mode, UMS mode offers the following advantages:

a. You can directly modify the configuration files in the /boot/ partition.

b. You can flash the system directly to the eMMC, just as you would write to a MicroSD card.

c. You can erase all data on the eMMC through software.

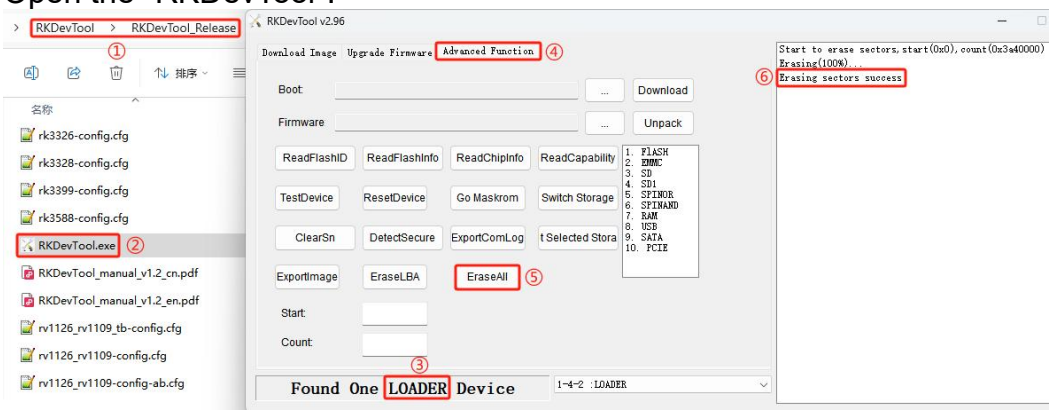
1. Follow the steps in “5.3.2.1 Using RKDevTool to Write the eMMC” to connect the board to the computer.

2. The computer will recognize the eMMC as a UMS device.

3. Install SD Card Formatter and use it to format the eMMC UMS device. (Do not use the built-in Windows formatting tool, as it cannot completely erase the data stored on the eMMC.)

5.4.2 Using RKDevTool to Erase eMMC (Windows)

1. Refer to the steps in "5.3.2.1 Using RKDevTool to Write the eMMC (Windows)" to connect the motherboard to the computer.
2. Open the "RKDevTool".



① Find the path where the downloaded RKDevTool is located.
② Open the RKDevTool.
③ The software will recognize a "LOADER" device. If it recognizes "MASKROOM," it indicates there is no data in the eMMC, hence no erase operation is necessary.
④ Click "Advanced Function."
⑤ Click "EraseAll" to begin erasing data from the eMMC.
⑥ "Erasing sectors success" indicates the erasure is complete.

5.4.3 Erasing eMMC After Booting from MicroSD Card

1. Refer to the steps in "5.3.2.2 Writing System onto eMMC Using a MicroSD Card" and log into the system terminal.

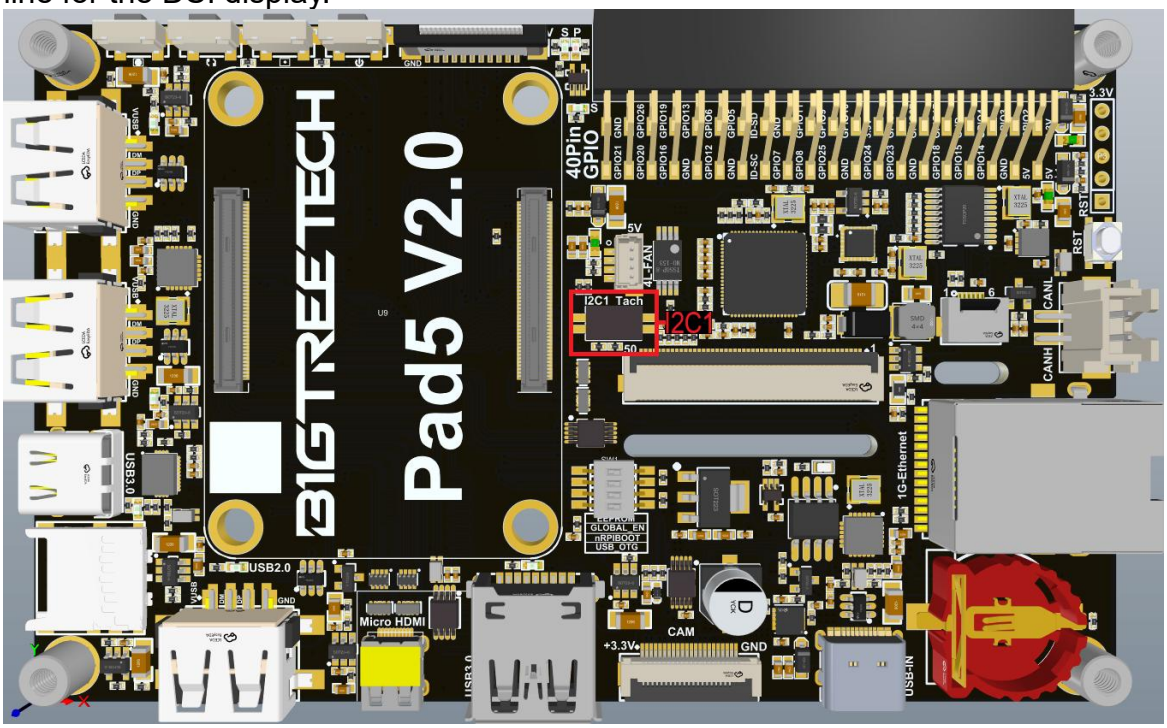
2. Run the command sudo mkfs /dev/mmcblk1 and then enter "y" to confirm.

```
biqu@BTT-CB2:~$ sudo mkfs /dev/mmcblk1
mke2fs 1.46.2 (28-Feb-2021)
/dev/mmcblk1 contains a ext2 file system
        created on Wed Apr 24 06:30:21 2024
Proceed anyway? (y,N) y
Discarding device blocks: done
Creating filesystem with 7634944 4k blocks and 1908736 inodes
Filesystem UUID: 51dbd34e-8aef-4f29-9f98-e535341ed141
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

biqu@BTT-CB2:~$
```

5.5. System Configuration

5.5.1 DSI Display

1. Set both 2-pin I2C DIP switches to ON. This interface carries the touch signal line for the DSI display.



2. Add dsi to the `overlays` value in `/boot/armbianEnv.txt`. Separate multiple overlays with spaces.
   The system includes `hdmi` in the default configuration; to enable both screens, keep both hdmi and dsi, and to use only one screen, keep only the desired display configuration and remove the other.

   After restarting, the configuration will take effect. If both HDMI and external DSI displays are configured, both displays should show the same content.

Important: The DSI touch I2C signal line on this device differs from the standard BIGTREETECH Pi2. The current system image does not yet support this touch signal, so touch functionality on the DSI display is currently unavailable.

Touch input must be remapped for multi-display setups.

2.1. Set the display:
export DISPLAY=:0.0

2.2. Query display names:

xrandr -q

Example output:

Screen 0: minimum 320 x 200, current 1600 x 480, maximum 16384 x 16384

HDMI-1 connected primary 800x480+0+0 (normal left inverted right x axis y axis) 108mm x 62mm

800x480 68.35*+ 68.35

640x480 60.00 59.94

DSI-1 connected 800x480+800+0 (normal left inverted right x axis y axis) 0mm x 0mm

800x480 56.06*+

This indicates that HDMI- and DSI-1 are available.

2.3. Query touch devices:

xinput list

Example output:

BIQU BTT-HDMI5 id=9 [slave pointer (2)]

This indicates that the touch device BIQU BTT-HDMI5 with ID 9 is available.

2.4. Run:
xinput map-to-output 9 HDMI-1

This maps touch device ID 9 to the HDMI output. The CB2 supports mirrored dual-display output only, meaning the HDMI screen acts as the primary display while the DSI screen mirrors the HDMI output. Therefore, all touch devices should be mapped to HDMI.

2.5. This command does not persist after reboot. Add it to a startup script if persistent mapping is required.

5.5.2 CAM Camera

1. Add `csi ` to the `overlays` value in ` /boot/armbianEnv.txt `, separating multiple overlays with spaces, then restart the device; the current CB2 system image supports only the ` ov5647` and `imx219` CSI camera drivers, and the system will automatically detect the camera type, so no additional configuration is required.
2. After booting, run:

   ```
   dmesg | grep csi
   dmesg | grep ov5647
   dmesg | grep imx219
   ```

   Check whether the CSI camera has been detected correctly.

5.5.3 RTC

1. The onboard RTC chip is `PCF8563`. A `CR1220` coin cell battery must be installed for the RTC to function when power is disconnected.
2. Add `rtc_pcf8563` to the `overlays` value in `/boot/armbianEnv.txt`, separating multiple overlays with spaces, then restart the device.
3. After booting, run:

   ```
   dmesg | grep rtc
   ```

   Check which RTC device number has been assigned. If `pcf8563` is mounted as `rtc0`, use `rtc0` in the following commands. If it is mounted as `rtc1`, use `rtc1` instead.
4. RTC Test Example: Using rtc0 as an example.

   Write the system time to RTC:

   ```
   sudo hwclock -w -f /dev/rtc0
   ```

   Set the system time from RTC:

   ```
   sudo hwclock -s -f /dev/rtc0
   ```

   Read and display the RTC time:

   ```
   sudo hwclock -r -f /dev/rtc0
   ```

6. Used with the BIGTREETECH CB1

6.1. Used with the BIGTREETECH CB1

Only the system images provided by BIGTREETECH are supported:
https://github.com/bigtreetech/CB1/releases

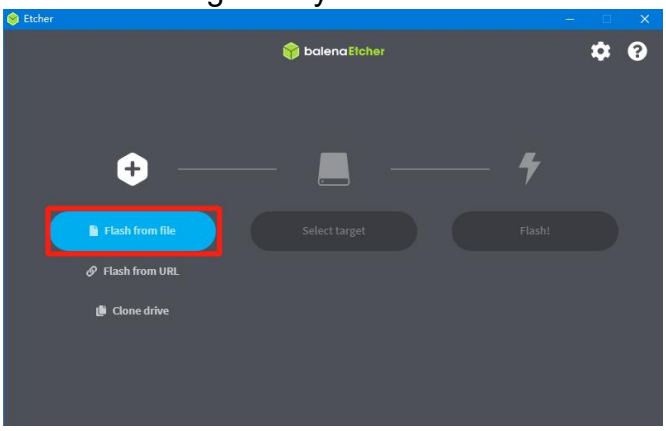
6.2. Download and Install balenaEtcher

https://www.balena.io/etcher/

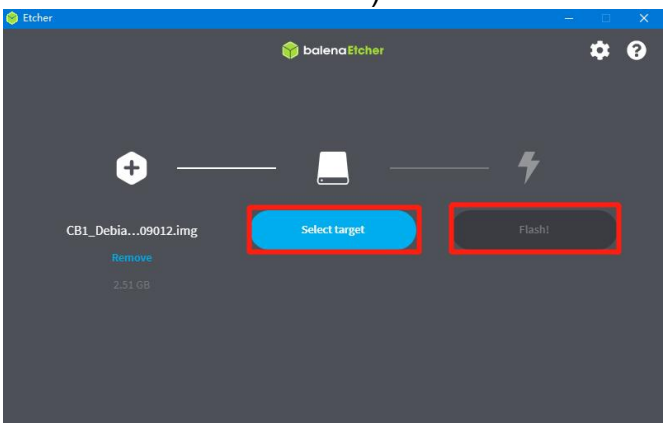
6.3. Flashing the System

6.3.1 Flashing the System

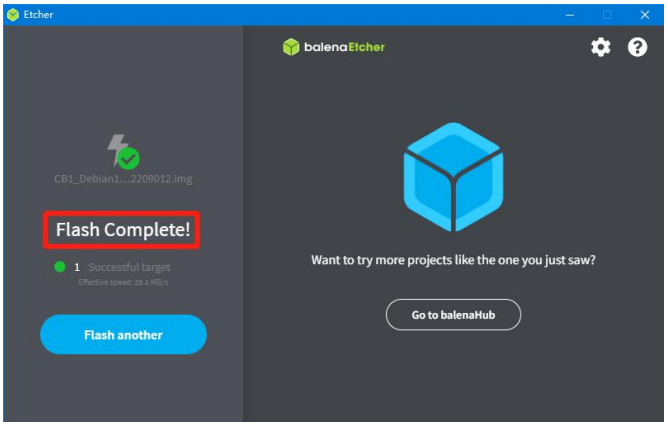
1. Insert a MicroSD card to your computer through a card reader.
2. Select the image that you downloaded.



3. Select the MicroSD card and click "WRITE" (WRITE the image will format the MicroSD card. Be careful not to select the wrong storage device, otherwise the data will be formatted.)



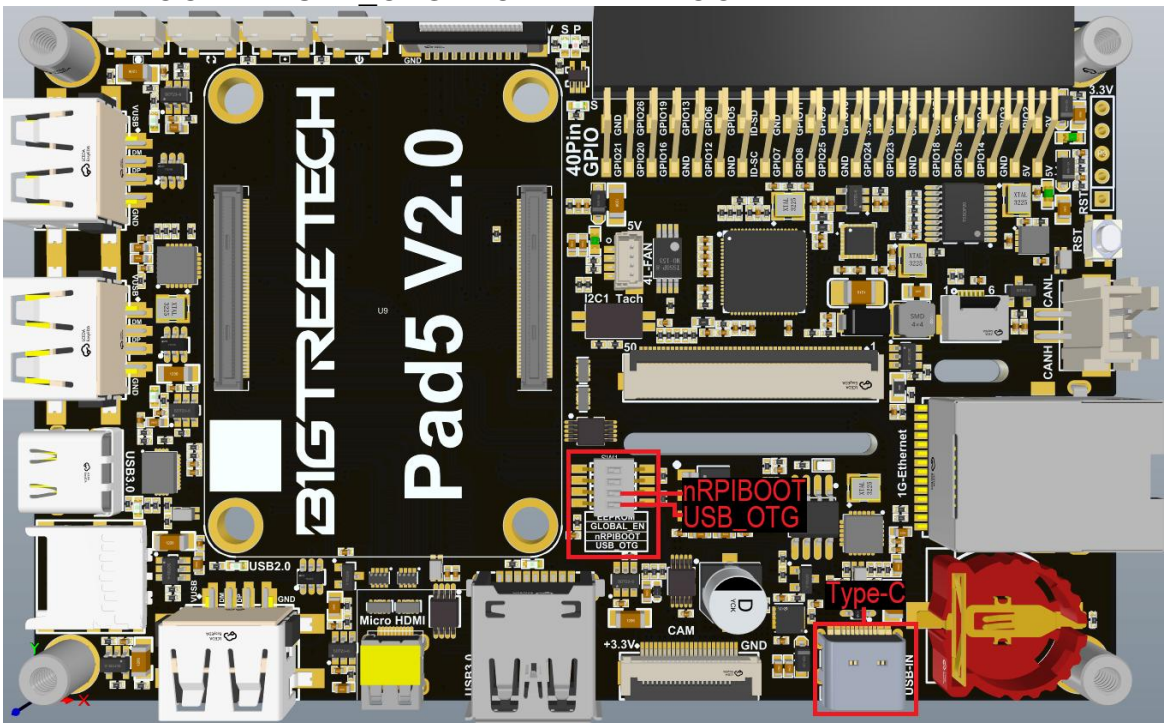
4. Wait for the writing to finish.



6.3.2 Flashing to eMMC

6.3.2.1 Use balenaEtcher to flash the system image to eMMC

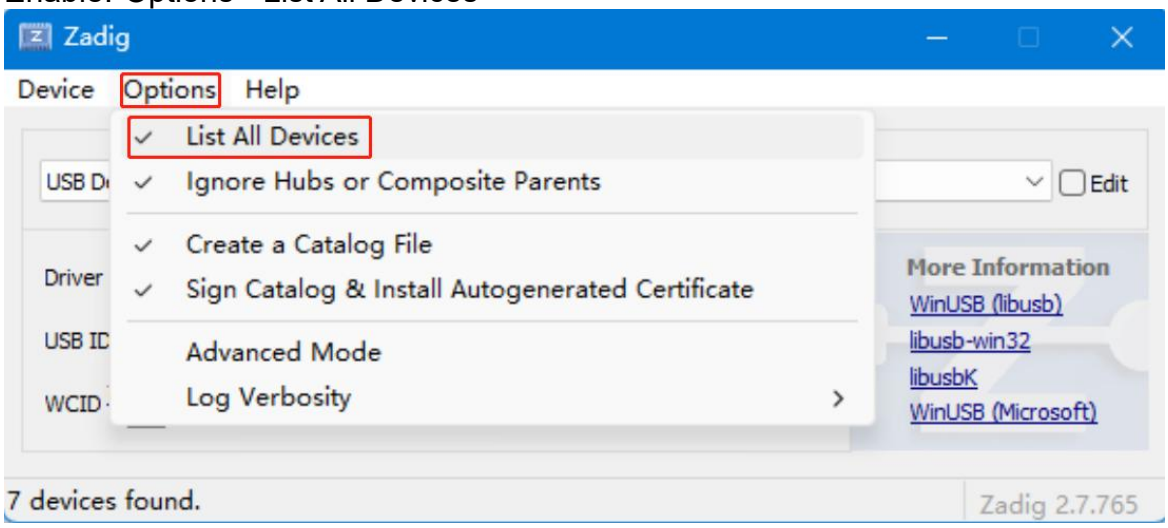
1. Download the sunxi-fel utility (Mac OS is not supported) and place it in the same directory as CB1 eMMC uboot file.
2. Set nRPIBOOT and USB_OTG to ON to enter BOOT mode.



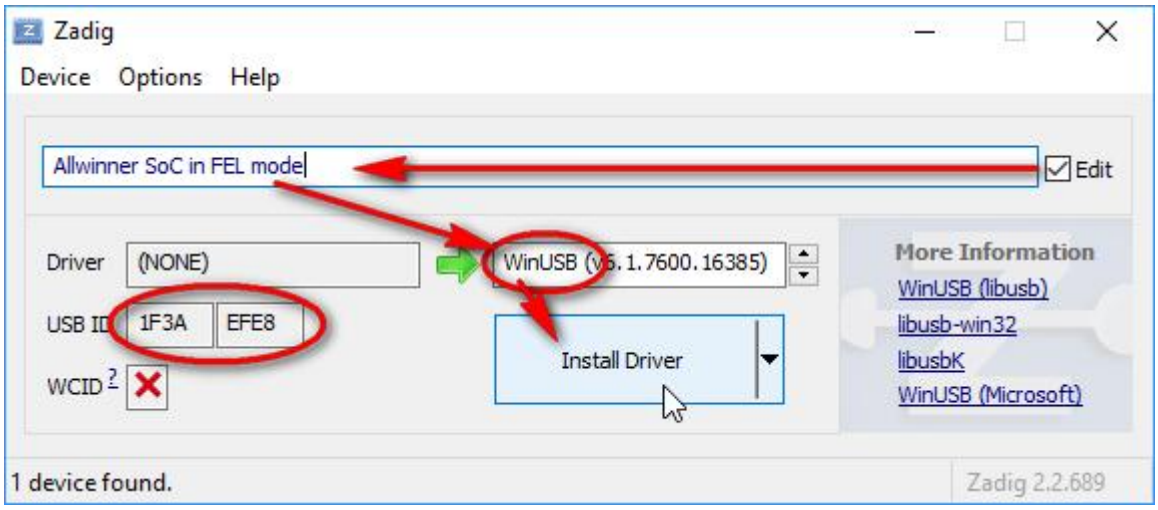
3. Connect the Type-C port to the computer's USB port. To avoid insufficient USB power, it is recommended to use a powered USB hub or provide external power through the 5V pins on the 40-pin header.
4. Linux includes the required driver by default. Windows requires an additional driver, which can be installed according to AllWinner's instructions.

   4.1. Download Zadig

4.2. Enable:`Options->List All Devices`



4.3. Select the CB1 USB device. In most cases it appears as an `unknown device` with USB ID `1F3A:EFE8`. After confirming the correct device is selected, click `Install Driver`.



5. Open `PowerShell (Windows)` or a `console terminal (Linux)` in the directory containing the sunxi-fel utility.

6. Run the following command to check whether USB communication is working.

Windows

```
.\sunxi-fel.exe -v ver
```

linux-armhf

```
sudo ./sunxi-fel-armhf -v ver
```

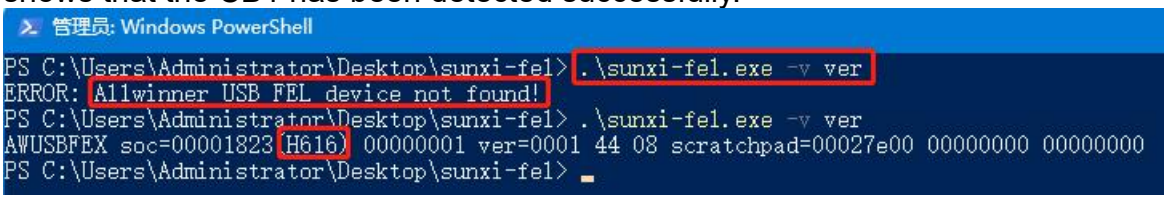
linux-aarch64

```
sudo ./sunxi-fel-aarch64 -v ver
```

If the output shows: `ERROR: Allwinner USB FEL device not found!`, it indicates that the correct USB device was not detected. Check the connection and driver installation.

If the output shows: `AWUSBFEX soc=00001823(H616)`, it indicates that the CB1 device has been detected correctly.

As shown in the image below, the result of the first command indicates that the CB1 device is not connected, while the result of the second command shows that the CB1 has been detected successfully.



7. Run the following command to write u-boot to the CB1.

   Windows

   .\sunxi-fel.exe uboot .\u-boot-sunxi-cb1-emmc.bin

   linux-armhf

   sudo ./sunxi-fel-armhf uboot ./u-boot-sunxi-cb1-emmc.bin

   linux-aarch64

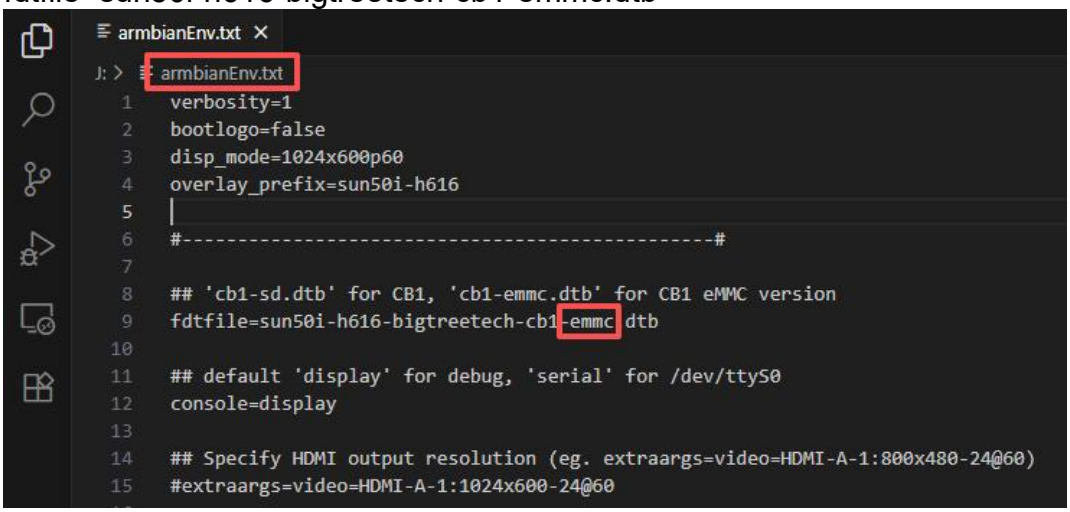
   sudo ./sunxi-fel-aarch64 uboot ./u-boot-sunxi-cb1-emmc.bin

After u-boot is written successfully, the eMMC will mount as a USB storage device on the computer.

8. Use balenaEtcher to flash the system image. The process is the same as flashing to a Micro SD card.

9. After flashing the CB1 eMMC version, an additional configuration change is required. Open `/BOOT/armbianEnv.txt` (or `/BOOT/BoardEnv.txt` on older versions) and change fdtfile from the default SD version to the eMMC version:

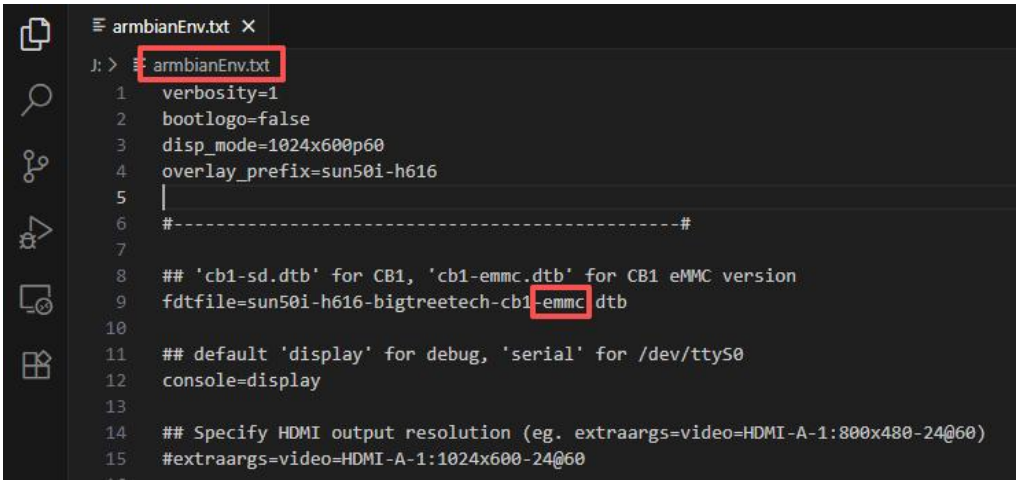
fdtfile=sun50i-h616-bigtreetech-cb1-emmc.dtb



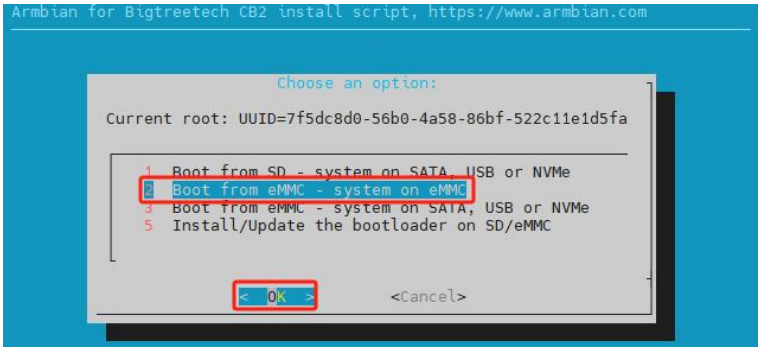
10. After saving the configuration, power off the device and switch nRPIBOOT and USB_OTG back to OFF. Power on the device again to enter normal operating mode.

6.3.2.2 Writing the System to eMMC from a Micro SD Card

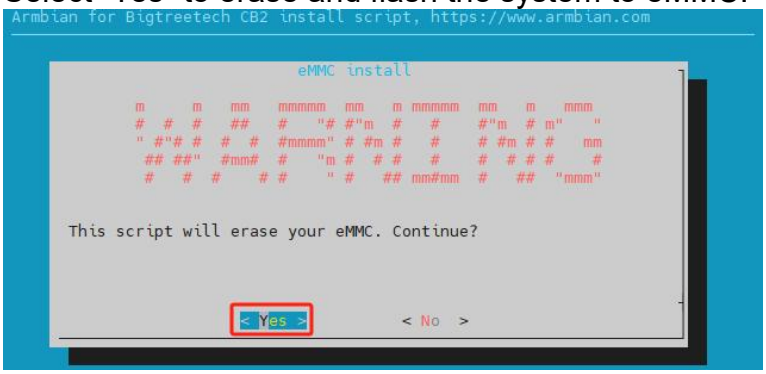
1. Flash the system image to a Micro SD card, then insert the card into the board and wait for the system to boot.
2. Connect to the system terminal through Ethernet, Wi-Fi, or USB-to-UART, and log in:
   login: biqu
   password: biqu
   (1) Because the CB1 eMMC version uses a different configuration from the non-eMMC version, modify the configuration before writing the system from the SD card to eMMC. Open: `/BOOT/armbianEnv.txt`(or `/BOOT/BoardEnv.txt`on older versions), change fdtfile from the SD version to the eMMC version:fdtfile=sun50i-h616-bigtreetech-cb1-emmc.dtb



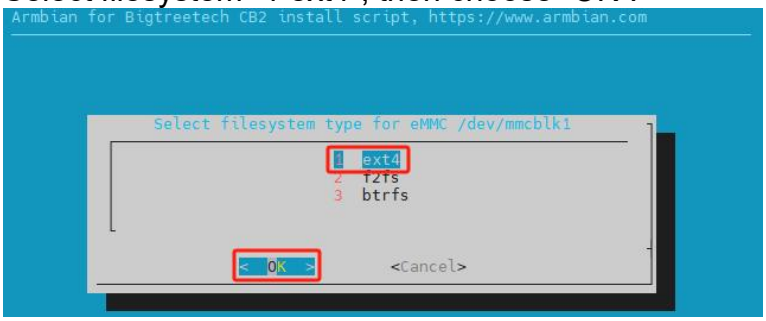
   (2) After saving the configuration, immediately write the modified system to eMMC. Once the SD card system is changed to the eMMC configuration, it will no longer boot from the SD card unless the setting is changed back.
   (3) Run: `sudo nand-sata-install `. In the menu, select: `2 Boot From eMMC - system on eMMC`, then choose `OK`.



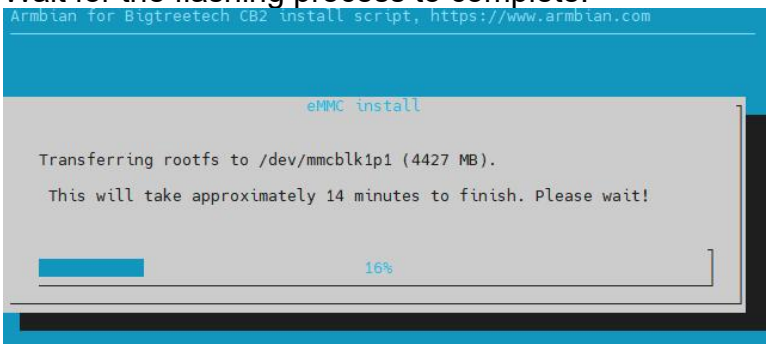
(4) Select `Yes` to erase and flash the system to eMMC.



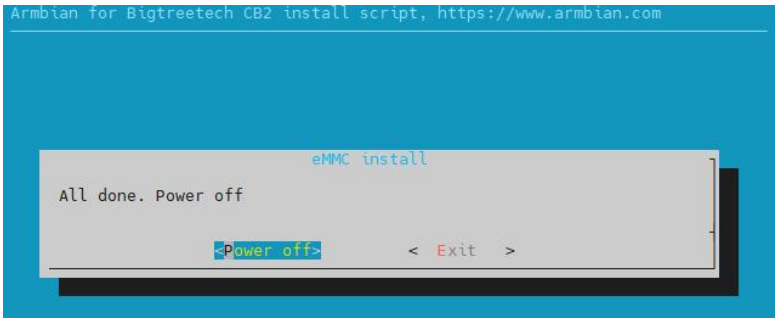
(5) Select filesystem `1 ext4`, then choose `OK`.



(6) Wait for the flashing process to complete.



(7) When flashing is complete, a prompt will ask whether to shut down. Select `Power off`.



(8) After shutdown, disconnect the power, remove the Micro SD card, and power on the device again. The system will now boot from eMMC.

6.4. Erasing eMMC

When using a Micro SD card as the system storage instead of eMMC, it is recommended to erase the eMMC data to prevent the board from accidentally booting from eMMC.

6.4.1 Erase eMMC Using UMS (Windows)

1. Follow the same procedure described above for mounting eMMC as a USB drive when flashing with balenaEtcher.。
2. Install SD Card Formatter and use it to format the eMMC UMS device(Do not use the default Windows format function, as it cannot completely erase the eMMC data).

6.4.2 Erase eMMC After Booting from a Micro SD Card

Follow the same procedure described above for logging into the system after booting from a Micro SD card.

Run:` sudo mkfs /dev/mmcblk1 `, then enter `y` to confirm.

7. Notes

1. The onboard SPI-to-CAN controller uses pins from the 40-pin GPIO header, so it is not recommended to connect other devices to those same pins. SPI pins used:

CS - GPIO8
MISO - GPIO9
MOSI - GPIO10
SCK - GPIO11
INT - GPIO24

2. On the Raspberry Pi CM5, both DSI and CSI can use the same pin output. Therefore, when using CM5, the onboard DSI and CAM (CSI) interfaces can be configured freely as either DSI or CSI. This means the board can be configured for either two DSI displays or two CSI cameras.

a) CAM (CSI) uses port 0

i. Configure as a DSI display：

```
[cm5]
dtoverlay=vc4-kms-v3d
dtoverlay=vc4-kms-dsi-generic,dsi0
dtoverlay=edt-ft5406,i2c_csi_dsi0
```

ii. Configure as a CSI camera：

```
[cm5]
dtoverlay=ov5647,cam0
```

b) DSI uses port 1

i. Configure as a DSI display：
[cm5]
dtoverlay=vc4-kms-v3d
dtoverlay=vc4-kms-dsi-generic
dtoverlay=edt-ft5406,i2c_csi_dsi

ii. Configure as a CSI camera
[cm5]
dtoverlay=ov5647

3. For Raspberry Pi CM4/CM5 eMMC versions, the system runs only from eMMC and does not support booting from a Micro SD card, whereas BIGTREETECH CB1/CB2 eMMC versions still support Micro SD card booting, with the Micro SD card having higher boot priority.